```
    std::shared_ptr<T> wait_and_pop()    <-- ❸
    {
        std::unique_lock<std::mutex> lk(mut);
        data_cond.wait(lk,[this]{return !data_queue.empty();});    <-- ❹
        std::shared_ptr<T> res(
            std::make_shared<T>(std::move(data_queue.front())));
        data_queue.pop();
        return res;
    }

    bool try_pop(T& value)
    {
        std::lock_guard<std::mutex> lk(mut);
        if(data_queue.empty())
            return false;
        value=std::move(data_queue.front());
        data_queue.pop();
        return true;
    }

    std::shared_ptr<T> try_pop()
    {
        std::lock_guard<std::mutex> lk(mut);
        if(data_queue.empty())
            return std::shared_ptr<T>();    <-- ❺
        std::shared_ptr<T> res(
            std::make_shared<T>(std::move(data_queue.front())));
        data_queue.pop();
        return res;
    }

    bool empty() const
    {
        std::lock_guard<std::mutex> lk(mut);
        return data_queue.empty();
    }
};
```

The structure of the queue implementation shown in listing 6.2 is similar to the stack from listing 6.1, except for the call to `data_cond.notify_one()` in `push()` ❶ and the `wait_and_pop()` functions ❷, ❸. The two overloads of `try_pop()` are almost identical to the `pop()` functions from listing 6.1, except that they don't throw an exception if the queue is empty. Instead, they return either a `bool` value indicating whether a value was retrieved or a `NULL` pointer if no value could be retrieved by the pointer-returning overload ❺.